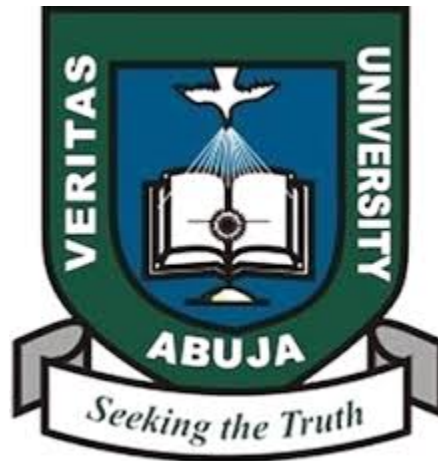


Department of Software Engineering
Veritas University, Bwari Abuja.



‘LAB MANUAL’

Course Code: VUA-SEN182

Course Title: “Web Programming (HTML, CSS, Git)”

Prepared by: Mr. Victor Ayamba

Course Contents

Contents	CLOs	Lecture	Practical
HTML: Basics, Elements, Attributes, Headings, Paragraphs, Formatting, Links, Head, Images, Tables, Lists, Blocks, Layout, Forms, Multimedia Tags, Semantic HTML, HTML5 Structure.	CLO – 1 & 2	4	8
CSS: Introduction, Syntax, Selectors (ID & Class), Styling Backgrounds, Text, Fonts, Links, Box Model (Border, Outline, Margin, Padding), Grouping/Nesting, Dimension, Display, Positioning, Floating, Align, Navigation Bar, Image Gallery, Flexbox, Grid Layout, Responsive Design.	CLO – 1 & 2	3	6
JavaScript: Introduction, Output, Statements, Comments, Variables, Data Types, Objects, Functions, Operators, Comparisons, Conditions, Loops, Error Handling, DOM Introduction and Manipulation, Number, String, Date, Array, Window, Screen, Navigator, Popup Alerts.	CLO – 1 & 2	4	8
Bootstrap: Introduction to Bootstrap Framework, Grid System, Containers and Layout, Responsive Design Principles, Typography and Utilities, Navigation Bars, Buttons, Cards, Forms and Input Groups, Tables, Alerts, Modals, Responsive Components and Layout Customization.	CLO – 1 & 2	4	8
Total		15	30

INTRODUCTION TO THE COURSE:

The Internet and the World Wide Web have become fundamental components of modern society, influencing communication, education, business, and everyday life. As digital platforms continue to expand, the ability to design and develop functional and visually appealing web applications has become an essential skill for software engineers and technology professionals. Understanding how web technologies work together enables students to create interactive systems that can serve real-world needs.

This laboratory course provides a practical introduction to modern Web Programming using core client-side technologies such as **HTML, CSS, JavaScript, and the Bootstrap framework as well as version control tools like Git and Github**. Students will learn how to structure web pages using HTML, style and layout interfaces with CSS, enhance interactivity through JavaScript, and build responsive, mobile-friendly user interfaces using Bootstrap. Through hands-on laboratory exercises, students will progressively develop the skills required to design, implement, and maintain modern web applications.

The lab sessions are structured to guide students step-by-step through the development of a meaningful web project while introducing essential web development concepts and tools. Each experiment builds upon the previous one, enabling learners to gradually transform basic web pages into fully structured and responsive websites. At the end of each lab session, tasks are provided to reinforce understanding and encourage independent exploration. Students are advised to follow this lab manual sequentially and carefully adhere to the provided guidelines to gain maximum benefit from the course.

Lab 3: Review of Key HTML Terminologies and Concepts.

HTML Semantic Elements

A semantic element clearly describes its meaning to both the browser and the developer.

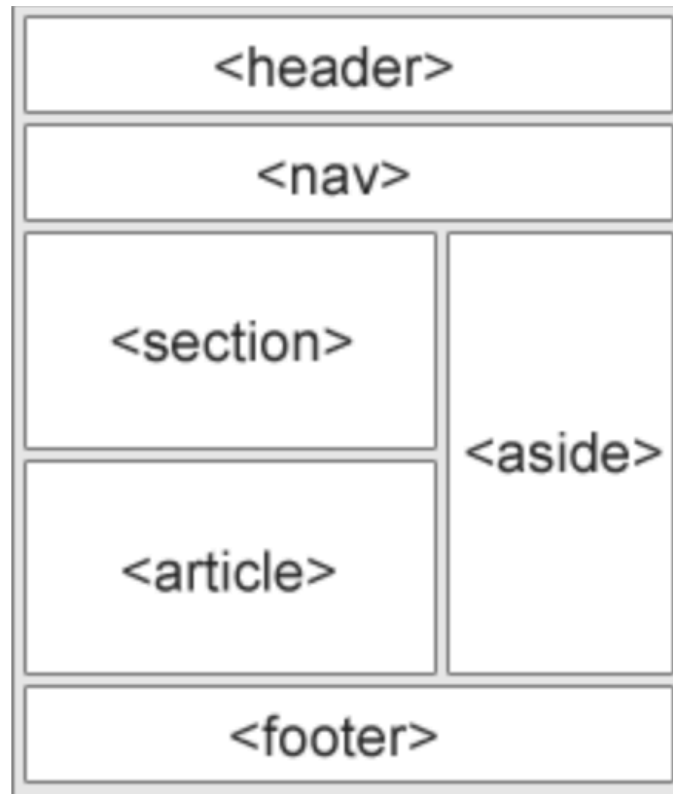
- Examples of non-semantic elements: `<div>` and `<span>` - Tells nothing about its content.
- Examples of semantic elements: `<img>`, `<table>`, and `<article>` - Clearly defines its content.

Many web sites contain HTML code like: `<div id="nav">` `<div class="header">` `<div id="footer">` to indicate navigation, header, and footer.

In HTML there are several semantic elements that can be used to define different parts of a web page:

Examples are:

- `<article>`
- `<aside>`
- `<details>`
- `<figcaption>`
- `<figure>`
- `<footer>`
- `<header>`
- `<main>`
- `<mark>`
- `<nav>`
- `<section>`
- `<summary>`
- `<time>`



HTML `<section>` Element

The `<section>` element defines a section in a document.

According to [W3C's HTML documentation](#): "A section is a thematic grouping of content, typically with a heading."

Examples of where a `<section>` element can be used:

- Chapters
- Introduction
- News items
- Contact information

A web page could normally be split into sections for introduction, content, and contact information.

Example

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Semantic Elements Example</title>
</head>
<body>
<section>
  <h2>Contact Information</h2>
  <p>Email: info@example.com</p>
  <p>Phone: +234 123 456 789</p>
</section>

</body>
</html>
```

HTML <article> Element

The <article> element specifies independent, self-contained content.

An article should make sense on its own, and it should be possible to distribute it independently from the rest of the web site.

Examples of where the <article> element can be used:

- Forum posts
- Blog posts
- User comments
- Product cards
- Newspaper articles

Example

```
<article>
  <h2>Latest News</h2>
  <p>Students are to resume on Monday.</p>
</article>
```

Nesting <article> in <section> or Vice Versa?

The <article> element specifies independent, self-contained content.

The <section> element defines a section in a document.

Can we use the definitions to decide how to nest those elements? No, we cannot!

So, you will find HTML pages with `<section>` elements containing `<article>` elements, and `<article>` elements containing `<section>` elements.

HTML `<header>` Element

The `<header>` element represents a container for introductory content or a set of navigational links.

A `<header>` element typically contains:

- one or more heading elements (`<h1>` - `<h6>`)
- logo or icon
- authorship information

Note: You can have several `<header>` elements in one HTML document. However, `<header>` cannot be placed within a `<footer>`, `<address>` or another `<header>` element.

Example

```
<header>
  <h1>Veritas University Portal</h1>
  <p>Welcome students and staff</p>
</header>
```

HTML `<footer>` Element

The `<footer>` element defines a footer for a document or section.

A `<footer>` element typically contains:

- authorship information
- copyright information
- contact information
- sitemap
- back to top links
- related documents

Example

```
<footer>
  <p>Contact us: support@example.com</p>
  <p>&copy; 2026 All rights reserved</p>
</footer>
```

You can have several `<footer>` elements in one document.

HTML `<nav>` Element

The `<nav>` element defines a set of navigation links.

Example

```
<nav>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">Courses</a></li>
    <li><a href="#">Login</a></li>
  </ul>
</nav>
```

Notice that NOT all links of a document should be inside a `<nav>` element. The `<nav>` element is intended only for major blocks of navigation links.

Browsers, such as screen readers for disabled users, can use this element to determine whether to omit the initial rendering of this content.

HTML `<aside>` Element

The `<aside>` element defines some content aside from the content it is placed in (like a sidebar).

The `<aside>` content should be indirectly related to the surrounding content.

Example

```
<aside>
  <h3>Related Topics</h3>
  <p>Check out our latest tutorials on HTML and CSS.</p>
</aside>
```

HTML <figure> and <figcaption> Elements

The `<figure>` tag specifies self-contained content, like illustrations, diagrams, photos, code listings, etc.

The `<figcaption>` tag defines a caption for a `<figure>` element. The `<figcaption>` element can be placed as the first or as the last child of a `<figure>` element.

Example

```
<figure>
  
  <figcaption>Figure 1: A beautiful natural scenery</figcaption>
</figure>
```



Figure 1: A beautiful natural scenery

The `<img>` element defines the actual image/illustration.

Why Semantic Elements?

According to the W3C: "A semantic Web allows data to be shared and reused across applications, enterprises, and communities."

HTML `<details>` and `<summary>` Elements

For instance, FAQ sections often use this to hide and show answers.

Example

`<details>`

```
<summary>Click to view more information</summary>
<p>This is hidden content that can be expanded.</p>
</details>
```

HTML `<mark>` Elements

For example, search results may highlight matched keywords using `<mark>`

Example

```
<p>This is a <mark>highlighted</mark> word in a sentence.</p>
```

HTML `<time>` Element

For instance, an event website may use `<time>` to display schedules in a machine-readable format.

Example

```
<p>Published on <time datetime="2026-03-20">March 20, 2026</time></p>
```

For further studies, refer to:

1. https://www.w3schools.com/html/html5_semantic_elements.asp
2. <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements>

HTML Block and Inline Elements

Every HTML element has a default display value, depending on what type of element it is.

The two most common display values are block and inline.

Block-level Elements

A block-level element always starts on a new line, and the browsers automatically add some space (a margin) before and after the element.

A block-level element always takes up the full width available (stretches out to the left and right as far as it can).

Two commonly used block elements are: `<p>` and `<div>`.

The `<p>` element defines a paragraph in an HTML document.

The `<div>` element defines a division or a section in an HTML document.

The `<p>` element is a block-level element.

The `<div>` element is a block-level element.

Here are the block-level elements in HTML:

- `<address>`
- `<article>`
- `<aside>`
- `<blockquote>`
- `<canvas>`
- `<dd>`
- `<div>`
- `<dl>`
- `<dt>`
- `<fieldset>`
- `<figcaption>`
- `<figure>`
- `<footer>`
- `<form>`
- `<h1>-<h6>`
- `<header>`
- `<hr>`
- `<li>`
- `<main>`
- `<nav>`
- `<noscript>`
- `<ol>`
- `<p>`
- `<pre>`
- `<section>`
- `<table>`
- `<tfoot>`
- `<ul>`
- `<video>`

Inline Elements

An inline element does not start on a new line. An inline element only takes up as much width as necessary. This is a `<span>` element inside a paragraph.

Here are the inline elements in HTML:

- `<a>`
- `<abbr>`
- `<acronym>`
- `<b>`
- `<bdo>`
- `<big>`
- `<br>`
- `<button>`
- `<cite>`
- `<code>`
- `<dfn>`
- `<em>`
- `<i>`
- `<img>`
- `<input>`
- `<kbd>`
- `<label>`
- `<map>`
- `<object>`
- `<output>`
- `<q>`
- `<samp>`
- `<script>`
- `<select>`
- `<small>`
- `<span>`
- `<strong>`
- `<sub>`
- `<sup>`
- `<textarea>`
- `<time>`
- `<tt>`
- `<var>`

The `<div>` Element

The `<div>` element is often used as a container for other HTML elements.

The `<div>` element has no required attributes, but `style`, `class` and `id` are common.

When used together with CSS, the `<div>` element can be used to style blocks of content:

`<div>` as a container

The `<div>` element is often used to group sections of a web page together.

```
<div>
  <h2>London</h2>
  <p>London is the capital city of England.</p>
  <p>London has over 9 million inhabitants.</p>
</div>
```

Center align a `<div>` element

If you have a `<div>` element that is not 100% wide, and you want to center-align it, set the CSS margin property to auto.

```
<style>
div {
  width:300px;
  margin:auto;
}
</style>
```

Aligning <div> elements side by side

When building web pages, you often want to have two or more `<div>` elements side by side, like this:

London	Oslo	Rome
London is the capital city of England.	Oslo is the capital city of Norway.	Rome is the capital city of Italy.
London has over 9 million inhabitants.	Oslo has over 700,000 inhabitants.	Rome has over 4 million inhabitants.

There are different methods for aligning elements side by side, all include some CSS styling. We will look at the most common methods:

Float

The CSS `float` property was not originally meant to align `<div>` elements side-by-side, but has been used for this purpose for many years.

The CSS float property is used for positioning and formatting content and allows elements to be positioned horizontally, rather than vertically.

```
<style>
.div1 {
  width:100%;
  overflow:auto;
}
.div2 {
  width:33%;
  float:left;
}
</style>
```

HTML class Attribute

The HTML `class` attribute is used to specify a class for an HTML element.

Multiple HTML elements can share the same class.

The class Attribute

The `class` attribute is often used to point to a class name in a style sheet. It can also be used by JavaScript to access and manipulate elements with the specific class name.

In the following example we have three `<div>` elements with a `class` attribute with the value of "city". All of the three `<div>` elements will be styled equally according to the `.city` style definition in the head section:

```
<!DOCTYPE html>
<html>
<head>
<style>
.city {
  background-color: tomato;
  color: white;
  border: 2px solid black;
  margin: 20px;
  padding: 20px;
}
</style>
</head>
<body>
<div class="city">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>
<div class="city">
  <h2>Paris</h2>
  <p>Paris is the capital of France.</p>
</div>
<div class="city">
  <h2>Tokyo</h2>
  <p>Tokyo is the capital of Japan.</p>
</div>
</body>
```

</html>

In the following example we have two `<span>` elements with a `class` attribute with the value of "note". Both `<span>` elements will be styled equally according to the `.note` style definition in the head section:

The Syntax For Class

To create a class; write a period (.) character, followed by a class name. Then, define the CSS properties within curly braces {}:

Multiple Classes

HTML elements can belong to more than one class.

To define multiple classes, separate the class names with a space, e.g. `<div class="city main">`. The element will be styled according to all the classes specified.

In the following example, the first `<h2>` element belongs to both the `city` class and also to the `main` class, and will get the CSS styles from both of the classes:

```
<h2 class="city main">London</h2>
<h2 class="city">Paris</h2>
<h2 class="city">Tokyo</h2>
```

Different Elements Can Share Same Class

Different HTML elements can point to the same class name. In the following example, both `<h2>` and `<p>` point to the "city" class and will share the same style:

```
<h2 class="city">Paris</h2>
<p class="city">Paris is the capital of France</p>
```

HTML Buttons

Buttons let users interact with a web page. They can submit forms, run JavaScript, or trigger different actions when clicked. The HTML `<button>` element defines a clickable button. By itself, the button does nothing until you add an action to it.

Styling HTML Buttons

Buttons are often styled with CSS:

Disabled Buttons

Use the `disabled` attribute to make a button unclickable:

Button with JavaScript

You can run JavaScript when the user clicks a button using the `onclick` attribute:

Button Types

The `type` attribute defines what a button does when clicked. There are three button types:

- `type="button"` - A normal clickable button (does nothing by default)
- `type="submit"` - Submits a form
- `type="reset"` - Resets all form fields

Example

```
<button type="button">Normal Button</button>
<button type="submit">Submit</button>
<button type="reset">Reset</button>
```


HTML Iframes

An HTML iframe is used to display a web page within a web page.

HTML Iframe Syntax

The HTML `<iframe>` tag specifies an inline frame.

An inline frame is used to embed another document within the current HTML document.

Example

```
<iframe src="url" title="description"></iframe>
```

Or you can add the `style` attribute and use the CSS `height` and `width` properties:

Example

```
<iframe src="demo_iframe.htm" style="height:200px;width:300px;"  
title="Iframe Example"></iframe>
```

Iframe - Remove the Border

By default, an iframe has a border around it. To remove the border, add the `style` attribute and use the CSS `border` property:

Example

```
<!DOCTYPE html>  
<html>  
<body>  
  
<h2>Iframe - Target for a Link</h2>  
  
<iframe src="demo_iframe.htm" name="iframe_a" height="300px" width="100%"  
title="Iframe Example"></iframe>  
  
<p><a href="lab3.html" target="iframe_a">LABS 3</a></p>  
  
<p>When the target attribute of a link matches the name of an iframe, the  
link will open in the iframe.</p>  
  
</body>  
</html>
```

Note that : Not everyone allows their websites to be embedded as **iframes** on other websites.

SUMMARY

HTML is inexhaustible, we will cover more contents as we progress with practical implementations. Do not fail to reference these websites for further studies as you progress through

1. <https://www.w3schools.com/html>
2. <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements>

Lab 3: Practical – Building a Semantic HTML Web Page

SECTION B: Lab Tasks on Semantic Web Pages

Objective

At the end of this lab, students should be able to:

- Structure a complete HTML page using semantic elements
- Organize content into meaningful sections
- Apply elements like `<article>`, `<aside>`, `<figure>`, `<details>`, etc.
- Understand how semantic HTML improves readability and accessibility

Instructions

You are required to **build a student portal webpage step-by-step**, implementing each semantic concept as you progress.

For instance, a typical university portal may contain navigation menus, news updates, course sections, and contact information all structured using semantic HTML.

Lab Tasks

Task 1: Create the Basic HTML Structure

What to do

Create a standard HTML5 document structure.

Requirement

- Include `<!DOCTYPE html>`
- Add `<html>`, `<head>`, and `<body>`
- Set the page title to **"Student Portal"**

Task 2: Add a `<header>` Section

What to do

Create a header that introduces the webpage.

Requirement

- Add a main heading (`<h1>`)
- Add a welcome message
- Include the current date using `<time>`

Task 3: Create Navigation using `<nav>`

What to do

Add a navigation menu.

Requirement

- Use `<nav>`
- Include at least 4 links:
 - Introduction
 - News
 - Courses
 - Contact

Task 4: Create the Main Content Area

What to do

Wrap all main content inside `<main>`.

Requirement

- All sections and articles must be inside `<main>`

Task 5: Create an Introduction Section using `<section>`

What to do

Add an introduction section.

Requirement

- Use `<section>`
- Add a heading
- Add a paragraph describing the portal

Task 6: Add an `<aside>` (Sidebar Content)

What to do

Include related information beside your introduction.

Requirement

- Use `<aside>`
- Add helpful or related content
- Highlight an important word using `<mark>`

Task 7: Create a News Section with `<article>`

What to do

Add a section for news updates.

Requirement

- Use `<section>`
- Inside it, create **at least two** `<article>` elements
- Each article must include:
 - A heading
 - A paragraph
 - A `<footer>` with date using `<time>`

Task 8: Nesting Practice (`<section>` and `<article>`)

What to do

Ensure proper nesting of elements.

Requirement

- Place `<article>` inside `<section>`
- Each article should be independent but grouped under one section

Task 9: Add a Course Section with `<figure>`

What to do

Display a course with an image.

Requirement

- Use `<figure>`
- Add an `<img>`
- Add `<figcaption>` describing the image

Task 10: Create FAQ using `<details>` and `<summary>`

What to do

Add a FAQ section.

Requirement

- Use at least **two** `<details>` elements
- Each must contain:
 - `<summary>` (question)
 - `<p>` (answer)

Task 11: Add a Footer using `<footer>`

What to do

Create the page footer.

Requirement

- Include:
 - Contact information
 - Copyright
 - A “Back to Top” link

Task 12: Use Semantic Elements Properly

What to do

Review your work.

Checklist

- Avoid excessive `<div>` usage
- Use semantic tags appropriately

- Ensure proper nesting
- Ensure each section has a heading

Final Task (Integration Challenge)

What to do

Combine all tasks into **one complete, well-structured webpage**.

Expectation

Your final page should include:

- Header
- Navigation
- Multiple sections
- Articles
- Sidebar (**aside**)
- Image with caption
- FAQ (collapsible content)
- Footer